Document Number: P0482R6
Date: 2018-11-09
Audience: Core Working Group
Library Working Group
Reply-to: Tom Honermann <tom@honermann.net>

char8_t: A type for UTF-8 characters and strings (Revision 6)

Changes since P0482R5

- Addressed CWG review feedback:
  - Updated the feature test macro values to 201811 and added a drafting note that the final value will be chosen by the project editor to reflect the date of approval.
- Addressed LWG review feedback:
  - Updated the specification for `c8rtomb` in 20.5.6 [c.mb.wcs] to reflect that calls that pass a null pointer for argument `buf` may introduce a data race.
  - Changed a use of *char8_t string literal* to *UTF-8 string literal* in C.1.1 [diff.lex] paragraph 3 to align with C terminology.
  - Added an example of silent backward compatibility impact to C.5.? [input.output] concerning passing UTF-8 literals to ostream inserters.
  - Corrected missing monospace markup for `source` in D.?? [depr.fs.path.factory].

Introduction

C++11 introduced support for UTF-8, UTF-16, and UTF-32 encoded string literals via N2249 [N2249]. New `char16_t` and `char32_t` types were added to hold values of code units for the UTF-16 and UTF-32 variants, but a new type was not added for the UTF-8 variants. Instead, UTF-8 character literals (added in C++17 via N4197 [N4197]) and UTF-8 string literals were defined in terms of the `char` type used for the code unit type of ordinary character and string literals. UTF-8 is the only text encoding mandated to be supported by the C++ standard for which there is no distinct code unit type. Lack of a distinct type for UTF-8 encoded character and string literals prevents the use of overloading and template specialization in interfaces designed for interoperability with encoded text. The inability to infer an encoding for narrow characters and strings limits design possibilities and hinders the production of elegant interfaces that work seemlessly in generic code. Library authors must choose to limit encoding support,

design interfaces that require users to explicitly specify encodings, or provide distinct interfaces for, at least, the implementation defined execution and UTF-8 encodings.

Whether `char` is a signed or unsigned type is implementation defined and implementations that use an 8-bit signed char are at a disadvantage with respect to working with UTF-8 encoded text due to the necessity of having to rely on conversions to unsigned types in order to correctly process leading and continuation code units of multi-byte encoded code points.

The lack of a distinct type and the use of a code unit type with a range that does not portably include the full unsigned range of UTF-8 code units presents challenges for working with UTF-8 encoded text that are not present when working with UTF-16 or UTF-32 encoded text. Enclosed is a proposal for a new `char8_t` fundamental type and related library enhancements intended to remove barriers to working with UTF-8 encoded text and to enable generic interfaces that work with all five of the standard mandated text encodings in a consistent manner.

Motivation

Consider the following string literal expressions, all of which encode `U+0123`, `LATIN SMALL LETTER G WITH CEDILLA`:

```
u8"\u0123" // UTF-8:  const char[]:     0xC4 0xA3 0x00
 u"\u0123" // UTF-16: const char16_t[]: 0x0123 0x0000
 U"\u0123" // UTF-32: const char32_t[]: 0x00000123 0x00000000
  "\u0123" // ???:    const char[]:     ???
 L"\u0123" // ???:    const wchar_t[]:  ???
```

The UTF-8, UTF-16, and UTF-32 string literals have well-defined and portable sequences of code unit values. The ordinary and wide string literal code unit sequences depend on the implementation defined execution and execution wide encodings respectively. Code that is designed to work with text encodings must be able to differentiate these strings. This is straight forward for wide, UTF-16, and UTF-32 string literals since they each have a distinct code unit type suitable for differentiation via function overloading or template specialization. But for ordinary and UTF-8 string literals, differentiating between them requires additional information since they have the same code unit type. That additional information might be provided implicitly via differently named functions, or explicitly via additional function or template arguments. For example:

```
// Differentiation by function name:
void do_x(const char *);
void do_x_utf8(const char *);
void do_x(const wchar_t *);
void do_x(const char16_t *);
void do_x(const char32_t *);

// Differentiation by suffix for user-defined literals:
int operator ""_udl(const char *s, std::size_t);
int operator ""_udl_utf8(const char *s, std::size_t);
int operator ""_udl(const wchar_t *s, std::size_t);
int operator ""_udl(const char16_t *s, std::size_t);
int operator ""_udl(const char32_t *s, std::size_t);

// Differentiation by function parameter:
void do_x2(const char *, bool is_utf8);
void do_x2(const wchar_t *);
void do_x2(const char16_t *);
void do_x2(const char32_t *);

// Differentiation by template parameter:
template<bool IsUTF8>
void do_x3(const char *);
```

The requirement to, in some way, specify the text encoding, other than through the type of the string, limits the ability to provide elegant encoding sensitive interfaces. Consider the following invocations of the `make_text_view` function proposed in P0244R2 [P0244R2].

```
make_text_view<execution_character_encoding>("text")
make_text_view<execution_wide_character_encoding>(L"text")
make_text_view<utf8_encoding>(u8"text")
make_text_view<utf16_encoding>(u"text")
make_text_view<utf32_encoding>(U"text")
```

For each invocation, the encoding of the string literal is known at compile time, so having to explicitly specify the encoding tag is

redundant. If UTF-8 string literals had a distinct type, then the encoding type could be inferred, while still allowing an overriding tag to be supplied:

```
make_text_view("text")   // defaults to execution_character_encoding.
make_text_view(L"text")  // defaults to execution_wide_character_encoding.
make_text_view(u8"text") // defaults to utf8_encoding.
make_text_view(u"text")  // defaults to utf16_encoding.
make_text_view(U"text")  // defaults to utf32_encoding.
make_text_view<utf16be_encoding>("\0t\0e\0x\0t\0")  // Default overridden to select UTF-16BE.
```

The inability to infer an encoding for narrow strings doesn't just limit the interfaces of new features under consideration. Compromised interfaces are already present in the standard library.

Consider the design of the `codecvt` class template. The standard specifies the following specializations of `codecvt` be provided to enable transcoding text from one encoding to another.

```
codecvt<char, char, mbstate_t>     // #1
codecvt<wchar_t, char, mbstate_t>  // #2
codecvt<char16_t, char, mbstate_t> // #3
codecvt<char32_t, char, mbstate_t> // #4
```

#1 performs no conversions. #2 converts between strings encoded in the implementation defined wide and narrow encodings. #3 and #4 convert between either the UTF-16 or UTF-32 encoding and the UTF-8 encoding. Specializations are not currently specified for conversion between the implementation defined narrow and wide encodings and any of the UTF-8, UTF-16, or UTF-32 encodings. However, if support for such conversions were to be added, the desired interfaces are already taken by #1, #3 and #4.

The file system interface adopted for C++17 via P0218R1 [P0218R1] provides an example of a feature that supports all five of the standard mandated encodings, but does so with an asymetric interface due to the inability to overload functions for UTF-8 encoded strings. Class `std::filesystem::path` provides the following constructors to initialize a `path` object based on a range of code unit values where the encoding is inferred based on the value type of the range.

```
template <class Source>
path(const Source& source);
template <class InputIterator>
path(InputIterator first, InputIterator last);
```

§ 30.11.7.2.2 [fs.path.type.cvt] describes how the source encoding is determined based on whether the source range value type is `char`, `wchar_t`, `char16_t`, or `char32_t`. A range with value type `char` is interpreted using the implementation defined execution encoding. It is not possible to construct a path object from UTF-8 encoded text using these constructors.

To accommodate UTF-8 encoded text, the file system library specifies the following factory functions. Matching factory functions are not provided for other encodings.

```
template <class Source>
path u8path(const Source& source);
template <class InputIterator>
path u8path(InputIterator first, InputIterator last);
```

The requirement to construct `path` objects using one interface for UTF-8 strings vs another interface for all other supported encodings creates unnecessary difficulties for portable code. Consider an application that uses UTF-8 as its internal encoding on POSIX systems, but uses UTF-16 on Windows. Conditional compilation or other abstractions must be implemented and used in otherwise platform neutral code to construct `path` objects.

The inability to infer an encoding based on string type is not the only challenge posed by use of `char` as the UTF-8 code unit type. The following code exhibits implementation defined behavior.

```
bool is_utf8_multibyte_code_unit(char c) {
  return c >= 0x80;
}
```

UTF-8 leading and continuation code units have values in the range 128 (0x80) to 255 (0xFF). In the common case where `char` is implemented as a signed 8-bit type with a two's complement representation and a range of -128 (-0x80) to 127 (0x7F), these

values exceed the unsigned range of the `char` type. Such implementations typically encode such code units as unsigned values which are then reinterpreted as signed values when read. In the code above, integral promotion rules result in `c` being promoted to type `int` for comparison to the `0x80` operand. if `c` holds a value corresponding to a leading or continuation code unit value, then its value will be interpreted as negative and the promoted value of type `int` will likewise be negative. The result is that the comparison is always false for these implementations.

To correct the code above, explicit conversions are required. For example:

```
bool is_utf8_multibyte_code_unit(char c) {
  return static_cast<unsigned char>(c) >= 0x80;
}
```

Finally, processing of UTF-8 strings is currently subject to an optimization pessimization due to glvalue expressions of type `char` potentially aliasing objects of other types. Use of a distinct type that does not share this aliasing behavior may allow for further compiler optimizations.

As of November 2017, UTF-8 is now used by more than 90% of all websites [W3Techs]. The C++ standard must improve support for UTF-8 by removing the existing barriers that result in redundant tagging of character encodings, non-generic UTF-8 specific workarounds like `u8path`, and the need for static casts to examine UTF-8 code unit values.

Proposal

The proposed changes are intended to bring the standard to the state the author believes it would likely be in had `char8_t` been added at the same time that `char16_t` and `char32_t` were added. This includes the ability to differentiate ordinary and UTF-8 literals in function overloading, template specializations, and user-defined literal operator signatures. The following core language changes are proposed in order to facilitate these capabilities:

- A new fundamental type named `char8_t`. This integral type has the same signedness, size, alignment, and integer conversion rank as `unsigned char`, but does not alias with any other type (e.g., this proposal does not add `char8_t` to the list of aliasing types in § 8.2.1 [basic.lval] paragraph 11 (11.8)).
- The type of UTF-8 string literals is changed from array of `const char` to array of `const char8_t`.
- The type of UTF-8 character literals is changed from `char` to `char8_t`.
- New `char8_t` based signatures for user-defined literal operators.

The following library changes are proposed to address concerns like those raised in the motivation section above, and to take advantage of the new core features:

- New `char8_t` based specializations of `atomic`, `numeric_limits`, `hash`, `char_traits`, `basic_string`, and `basic_string_view`.
- New `u8streampos`, `u8string`, `u8string_view` type aliases.
- New `operator ""s` and `operator ""sv` `char8_t` based overloads for UTF-8 literals.
- New `char8_t` based specializations of `codecvt` and `codecvt_byname` for converting between UTF-16, UTF-32, and UTF-8. The existing `char` based specializations are deprecated. The new specializations are functionally identical to the deprecated ones.
- The return type of the `u8string` and `generic_u8string` member functions of the filesystem `path` class are changed from `string` to `u8string`.
- Filesystem `path` objects may now be constructed with UTF-8 strings using the existing `path` constructors used for construction with other encodings. The existing `u8path` factory functions are deprecated.

These changes necessarily impact backward compatibility as described in the Backward compatibility section.

Design Considerations

Backward compatibility

This proposal does not specify any backward compatibility features other than to retain interfaces that it deprecates. The author believes such features are necessary, but that a single set of such features would unnecessarily compromise the goals of this proposal. Rather, the expectation is that implementations will provide options to enable more fine grained compatibility features.

The following sections discuss backward compatibility impact.

Core language backward compatibility

Initialization

Declarations of arrays of `char` may currently be initialized with UTF-8 string literals. Under this proposal, such initializations would become ill-formed. This is intended to maintain consistency with initialization of arrays of `wchar_t`, `char16_t`, and `char32_t`, all of which require the initializing string literal to have a matching element type as specified in § 11.6.2 [dcl.init.string].

```
char ca[] = u8"text";   // C++17: Ok.
                        // This proposal: Ill-formed.

char8_t c8a[] = "text"; // C++17: N/A (char8_t is not a type specifier).
                        // This proposal: Ill-formed.
```

Implementations are encouraged to add options to allow the above initializations (with a warning) to assist users in migrating their code.

Declarations of variables of type `char` initialized with a UTF-8 character literal remain well-formed and are initialized following the standard conversion rules.

```
char c = u8'c';         // C++17: Ok.
                        // This proposal: Ok (no change from C++17).

char8_t c8 = 'c';       // C++17: N/A (char8_t is not a type specifier).
                        // This proposal: Ok; c8 is assigned the value of the 'c'
                        //                character in the execution character set.
```

Implicit conversions

Under this proposal, UTF-8 string literals no longer bind to references to array of type `const char` nor do they implicitly convert to pointer to `const char`. The following code is currently well-formed, but would become ill-formed under this proposal:

```
const char (&u8r)[] = u8"text"; // C++17: Ok.
                                // This proposal: Ill-formed.

const char *u8p = u8"text";     // C++17: Ok.
                                // This proposal: Ill-formed.
```

Implementations are encouraged to add options to allow the above conversions (with a warning) to assist users in migrating their code. Such options would require allowing aliasing of `char` and `char8_t`. Note that it may be useful to permit these conversions only for UTF-8 string literals and not for general expressions of array of `char8_t` type.

Type deduction

Under this proposal, UTF-8 string and character literals have type array of `const char8_t` and `char8_t` respectively. This affects the types deduced for placeholder types and template parameter types.

```
template<typename T1, typename T2>
void ft(T1, T2);

ft(u8"text", u8'c'); // C++17: T1 deduced to const char*, T2 deduced to char.
                     // This proposal: T1 deduced to const char8_t*, T2 deduced to char8_t.

auto u8p = u8"text"; // C++17: Type deduced to const char*.
                     // This proposal: Type deduced to const char8_t*.

auto u8c = u8'c';    // C++17: Type deduced to char.
                     // This proposal: Type deduced to char8_t.
```

This change in behavior is a primary objective of this proposal. Implementations are encouraged to add options to disable `char8_t` support entirely when necessary to preserve compatibility with C++17.

Overload resolution

The following code is currently well-formed, and would remain well-formed under this proposal, but would behave differently:

```
template<typename T> void f(const T*);
void f(const char*);
f(u8"text");                    // C++17: Calls f(const char*).
                                // This proposal: Calls f<char8_t>(const char8_t*).
```

The following code is currently well-formed, but would become ill-formed under this proposal:

```
void f(const char*);
f(u8"text");                    // C++17: Ok.
                                // This proposal: Ill-formed; no matching function found.

int operator ""_udl(const char*, size_t);
auto x = u8"text"_udl;          // C++17: Ok
                                // This proposal: Ill-formed; no matching literal operator found.
```

These changes in behavior are a primary objective of this proposal. Implementations are encouraged to add options to disable `char8_t` support entirely when necessary to preserve compatibility with C++17.

Template specialization

The following code is currently well-formed, and would remain well-formed under this proposal, but would behave differently:

```
template<typename T> struct ct { static constexpr bool value = false; };
template<> struct ct<char> { static constexpr bool value = true; };
template<typename T> bool ft(const T*) { return ct<T>::value; }
ft(u8"text");                   // C++17: returns true.
                                // This proposal: returns false.
```

This change in behavior is a primary objective of this proposal. Implementations are encouraged to add options to disable `char8_t` support entirely when necessary to preserve compatibility with C++17.

Library backward compatibility

Return type of `path::u8string` and `path::generic_u8string`

This proposal includes a new specialization of `std::basic_string` for the new `char8_t` type, a new `std::u8string` type alias, and changes to the `u8string` and `generic_u8string` member functions of `filesystem::path` to return `std::u8string` instead of `std::string`. This change renders ill-formed the following code that is currently well-formed.

```
void f(std::filesystem::path p) {
  std::string s;

  s = p.u8string(); // C++17: Ok.
                    // This proposal: ill-formed.
}
```

Implementations are encouraged to add an option that allows implicit conversion of `std::u8string` to `std::string` to assist in a gradual migration of code that calls these functions.

Return type of `operator ""s` and `operator ""sv`

This proposal includes new overloads of `operator ""s` and `operator ""sv` that return `char8_t` specializations of `std::basic_string` and `std::basic_string_view` respectively. This change renders ill-formed the following code that is currently well-formed.

```
std::string s;

s = u8"text"s;    // C++17: Ok.
                  // This proposal: ill-formed.

s = u8"text"sv;   // C++17: Ok.
                  // This proposal: ill-formed.
```

Implementations are encouraged to add an option that allows implicit conversion of `std::u8string` to `std::string` to assist in a gradual migration of code that calls these functions.

Should UTF-8 literals continue to be referred to as narrow literals?

UTF-8 literals are maintained as narrow literals in this proposal.

What should be the underlying type of char8_t?

There are several choices for the underlying type of `char8_t`. Use of `unsigned char` closely aligns with historical use. Use of `uint_least8_t` would maintain consistency with how the underlying types of `char16_t` and `char32_t` are specified.

This proposal specifies `unsigned char` as the underlying type as noted in the changes to § 6.7.1 `[basic.fundamental]` paragraph 5.

Implementation Experience

An implementation is available in the `char8_t` branch of a gcc fork hosted on GitHub at https://github.com/tahonermann/gcc/tree/char8_t. This implementation is believed to be complete for both the proposed core language and library features with the exception of the proposed `mbrtoc8` and `c8rtomb` transcoding functions (the author expects to complete these shortly). New `-fchar8_t` and `-fno-char8_t` compiler options support enabling and disabling the new features. No backward compatibility features are currently implemented.

Richard Smith implemented support for the proposed core wording changes and they are present in the release of Clang 7. The changes are guarded by new `-fchar8_t` and `-fno-char8_t` options matching the gcc implementation. No backward compatibility features are currently implemented. Support for the proposed library features has not yet been implemented in libc++. Richard's changes can be found at http://llvm.org/viewvc/llvm-project?view=revision&revision=331244

Formal Wording

☐Hide deleted text

These changes are relative to N4762 [N4762]

Core wording

Change in table 5 of 5.11 [lex.key] paragraph 1:

> [...]
>
> Table 5 — Keywords
>
> | [...] |
> | --- |
> | char |
> | **char8_t** |
> | char16_t |
> | char32_t |
> | [...] |
>
> [...]

Change in 5.13.3 [lex.ccon] paragraph 3:

> A character literal that begins with `u8`, such as `u8'w'`, is a character literal of type ~~`char`~~**`char8_t`**, known as a *UTF-8 character literal*.[...]

Change in 5.13.5 [lex.string] paragraph 6:

> After translation phase 6, a *string-literal* that does not begin with an *encoding-prefix* is an *ordinary string literal*. **An ordinary string literal has type "*array* of *n* `const char`" where *n* is the size of the string as defined below, has static storage duration (6.6.4)**, and is initialized with the given characters.

Change in 5.13.5 [lex.string] paragraph 7:

> A *string-literal* that begins with `u8`, such as `u8"asdf"`, is a *UTF-8 string literal*, **also referred to as a `char8_t` string literal. A `char8_t` string literal has type "*array* of *n* `const char8_t`", where *n* is the size of the string as defined below; each successive element of the object representation (6.7) has the value of the corresponding code**

unit of the UTF-8 encoding of the string.

Change in 5.13.5 [lex.string] paragraph 8:

> Ordinary string literals and UTF-8 string literals are also referred to as narrow string literals. ~~A narrow string literal has type "*array* of *n* `const char`", where *n* is the size of the string as defined below, and has static storage duration (6.6.4).~~

Drafting note: The deleted paragraph 8 content was incorporated in the changes to paragraphs 6 and 7.

Remove 5.13.5 [lex.string] paragraph 9:

> ~~For a UTF-8 string literal, each successive element of the object representation (6.7) has the value of the corresponding code unit of the UTF-8 encoding of the string.~~

Drafting note: The paragraph 9 content was incorporated in the changes to paragraph 7.

Change in 5.13.5 [lex.string] paragraph 15:

> [...] In a narrow string literal, a *universal-character-name* may map to more than one `char` **or `char8_t`** element due to *multibyte encoding*. [...]

Change in 6.6.5 [basic.align] paragraph 6:

> The alignment requirement of a complete type can be queried using an `alignof` expression (7.6.2.6). Furthermore, the narrow character types (6.7.1) shall have the weakest alignment requirement. [*Note*: This enables the ~~narrow~~**ordinary** character types to be used as the underlying type for an aligned memory area (9.11.2). — *end note*]

Change in 6.7.1 [basic.fundamental] paragraph 1:

> Objects declared ~~as characters~~**with type** ~~(~~`char`~~)~~ shall be large enough to store any member of the implementation's basic character set. If a character from this set is stored in a character object, the integral value of that character object is equal to the value of the single character literal form of that character. It is implementation-defined whether a `char` object can hold negative values. Characters **declared with type `char`** can be explicitly declared `unsigned` or `signed`. Plain `char`, `signed char`, and `unsigned char` are three distinct types, collectively called ~~*narrow*~~***ordinary*** *character types*. **The ordinary character types and `char8_t` are collectively called *narrow character types*.** A `char`, a `signed char`, ~~and~~ an `unsigned char`**, and a `char8_t`** occupy the same amount of storage and have the same alignment requirements (6.6.5); that is, they have the same object representation. For narrow character types, all bits of the object representation participate in the value representation. [*Note*: A bit-field of narrow character type whose length is larger than the number of bits in the object representation of that type has padding bits; see 6.7. — *end note*] For unsigned narrow character types, each possible bit pattern of the value representation represents a distinct number. These requirements do not hold for other types. In any particular implementation, a plain `char` object can take on either the same values as a `signed char` or an `unsigned char`; which one is implementation-defined. For each value *i* ~~of type `unsigned char`~~ in the range 0 to 255 inclusive **of type `unsigned char` or `char8_t`**, there exists a value *j* of type `char` such that the result of an integral conversion (7.3.8) from *i* to `char` is *j*, and the result of an integral conversion from *j* to `unsigned char` **or `char8_t`** is *i*.

Change in 6.7.1 [basic.fundamental] paragraph 5:

> [...] Type `wchar_t` shall have the same size, signedness, and alignment requirements (6.6.5) as one of the other integral types, called its underlying type. **Type `char8_t` denotes a distinct type with the same size, signedness, and alignment as `unsigned char`, called its underlying type.** Types `char16_t` and `char32_t` denote distinct types with the same size, signedness, and alignment as `uint_least16_t` and `uint_least32_t`, respectively, in `<cstdint>`, called the underlying types.

Change in 6.7.1 [basic.fundamental] paragraph 7:

> Types `bool`, `char`, **`char8_t`,** `char16_t`, `char32_t`, `wchar_t`, and the signed and unsigned integer types are collectively called integral types. [...]

Change in 6.7.4 [conv.rank] subparagraph (1.8):

> [...]
> (1.8) — The ranks of **`char8_t`,** `char16_t`, `char32_t`, and `wchar_t` shall equal the ranks of their underlying types (6.7.1).
> [...]

Change to footnote 65 associated with 7.4 [expr.arith.conv] subparagraph (1.5):

> As a consequence, operands of type `bool`, **`char8_t,`** `char16_t`, `char32_t`, `wchar_t`, or an enumerated type are converted to some integral type.

Change in 7.6.2.3 [expr.sizeof] paragraph 1:

> [...] ~~`sizeof(char)`, `sizeof(signed char)` and `sizeof(unsigned char)` are 1~~**The result of `sizeof` applied to any of the narrow character types is 1**. The result of `sizeof` applied to any other fundamental type (6.7.1) is implementation-defined. [...]

Change in 9.1.7.2 [dcl.type.simple] paragraph 1:

> The simple type specifiers are
> *simple-type-specifier*:
> [...]
> `char`
> **`char8_t`**
> `char16_t`
> `char32_t`
> [...]

Change in table 11 of 9.1.7.2 [dcl.type.simple] paragraph 2:

> [...]
> Table 11 — *simple-type-specifiers* and the types they specify
>
> | Specifier(s) | Type |
> |---|---|
> | [...] | [...] |
> | `char` | "`char`" |
> | `unsigned char` | "`unsigned char`" |
> | `signed char` | "`signed char`" |
> | **`char8_t`** | **"`char8_t`"** |
> | `char16_t` | "`char16_t`" |
> | `char32_t` | "`char32_t`" |
> | [...] | [...] |
>
> [...]

Change in 9.3 [dcl.init] subparagraph (12.1):

> (12.1) — If an indeterminate value of unsigned ~~narrow~~**ordinary** character type (6.7.1) or `std::byte` type (16.2.1) is produced by the evaluation of:
> [...]
> (12.1.3) — the operand of a cast or conversion (7.3.8, 7.6.1.3, 7.6.1.9, 7.6.3) to an unsigned ~~narrow~~**ordinary** character type or `std::byte` type (16.2.1), or
> [...]
> then the result of the operation is an indeterminate value.

Change in 9.3 [dcl.init] subparagraph (12.2):

> (12.2) If an indeterminate value of unsigned ~~narrow~~**ordinary** character type or `std::byte` type is produced by the evaluation of the right operand of a simple assignment operator (7.6.18) whose first operand is an lvalue of unsigned ~~narrow~~**ordinary** character type or `std::byte` type, an indeterminate value replaces the value of the object referred to by the left operand.

Change in 9.3 [dcl.init] subparagraph (12.3):

> (12.3) If an indeterminate value of unsigned ~~narrow~~**ordinary** character type is produced by the evaluation of the initialization expression when initializing an object of unsigned ~~narrow~~**ordinary** character type, that object is initialized to an indeterminate value.

Change in 9.3 [dcl.init] subparagraph (12.4):

(12.4) If an indeterminate value of unsigned ~~narrow~~**ordinary** character type or `std::byte` type is produced by the evaluation of the initialization expression when initializing an object of `std::byte` type, that object is initialized to an indeterminate value.
[...]

Change in 9.3 [dcl.init] subparagraph (17.3):

[...]
(17.3) — If the destination type is an array of characters, **an array of `char8_t`,** an array of `char16_t`, an array of `char32_t`, or an array of `wchar_t`, and the initializer is a string literal, see 9.3.2.
[...]

Change in 9.3.2 [dcl.init.string] paragraph 1:

An array of ~~narrow~~**ordinary** character type (6.7.1), **`char8_t` array,** `char16_t` array, `char32_t` array, or `wchar_t` array can be initialized by ~~a narrow~~**an ordinary** string literal, **`char8_t` string literal,** `char16_t` string literal, `char32_t` string literal, or wide string literal, respectively, [...]

Change in 11.5.8 [over.literal] paragraph 3:

The declaration of a literal operator shall have a *parameter-declaration-clause* equivalent to one of the following:
[...]
`char`
`wchar_t`
`char8_t`
`char16_t`
`char32_t`
`const char*, std::size_t`
`const wchar_t*, std::size_t`
`const char8_t*, std::size_t`
`const char16_t*, std::size_t`
`const char32_t*, std::size_t`
[...]

Change in table 16 of 14.8 [cpp.predefined] paragraph 1:

Table 16 — Feature-test macros

Macro name	Value
[...]	[...]
__cpp_capture_star_this	201603L
__cpp_char8_t	**201811L *** placeholder *****
__cpp_constexpr	201603L
[...]	[...]

Drafting note: the final value for the `__cpp_char8_t` feature test macro will be selected by the project editor to reflect the date of approval.

Library wording

Change in 15.1 [library.general] paragraph 8:

The strings library (Clause 20) provides support for manipulating text represented as sequences of type `char`, **sequences of type `char8_t`,** sequences of type `char16_t`, sequences of type `char32_t`, sequences of type `wchar_t`, and sequences of any other character-like type.

Change in 15.3.2 [defns.character]:

[...]
[*Note:* The term does not mean only `char`, **`char8_t`,** `char16_t`, `char32_t`, and `wchar_t` objects, but any value that can be represented by a type that provides the definitions specified in these Clauses. — *end note*]

Change in table 35 of 16.3.1 [support.limits.general] paragraph 3:

Table 35 — Standard library feature-test macros

Macro name	Value	Header(s)
[...]	[...]	[...]
__cpp_lib_byte	201603L	<cstddef>
__cpp_lib_char8_t	**201811L ** *placeholder* ****	**<atomic> <filesystem> <istream> <limits> <locale> <ostream> <string> <string_view>**
__cpp_lib_chrono	201611L	<chrono>
[...]	[...]	[...]

Drafting note: the final value for the `__cpp_lib_char8_t` feature test macro will be selected by the project editor to reflect the date of approval.

Change in 16.3.2 [limits.syn]:

```
[…]
  template<> class numeric_limits<char>;
  template<> class numeric_limits<signed char>;
  template<> class numeric_limits<unsigned char>;
  template<> class numeric_limits<char8_t>;
  template<> class numeric_limits<char16_t>;
  template<> class numeric_limits<char32_t>;
  template<> class numeric_limits<wchar_t>;
[…]
```

Change in 20.2 [char.traits] paragraph 1:

> This subclause defines requirements on classes representing *character traits*, and defines a class template `char_traits<charT>`, along with ~~four~~**five** specializations, `char_traits<char>`, **`char_traits<char8_t>`,** `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`, that satisfy those requirements.

Change in 20.2 [char.traits] paragraph 4:

> This subclause specifies a class template, `char_traits<charT>`, and ~~four~~**five** explicit specializations of it, `char_traits<char>`, **`char_traits<char8_t>`,** `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`, all of which appear in the header `<string>` and satisfy the requirements below.

Drafting note: 20.2p4 appears to unnecessarily duplicate information previously presented in 20.2p1.

Change in 20.2.3 [char.traits.specializations]:

```
namespace std {
 template<> struct char_traits<char>;
 template<> struct char_traits<char8_t>;
 template<> struct char_traits<char16_t>;
 template<> struct char_traits<char16_t>;
 template<> struct char_traits<char32_t>;
 template<> struct char_traits<wchar_t>;
}
```

Change in 20.2.3 [char.traits.specializations] paragraph 1:

> The header `<string>` shall define ~~four~~**five** specializations of the class template `char_traits`: `char_traits<char>`, **`char_traits<char8_t>`,** `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`.

Add a new subclause after 20.2.3.1 [char.traits.specializations.char]:

> 20.2.3.? `struct char_traits<char8_t>` [char.traits.specializations.char8_t]
>
> ```
> namespace std {
>   template<> struct char_traits<char8_t> {
>     using char_type = char8_t;
>     using int_type = unsigned int;
>     using off_type = streamoff;
>     using pos_type = u8streampos;
> ```

```
  using state_type = mbstate_t;

  static constexpr void assign(char_type& c1, const char_type& c2) noexcept;
  static constexpr bool eq(char_type c1, char_type c2) noexcept;
  static constexpr bool lt(char_type c1, char_type c2) noexcept;

  static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
  static constexpr size_t length(const char_type* s);
  static constexpr const char_type* find(const char_type* s, size_t n,
                                         const char_type& a);
  static char_type* move(char_type* s1, const char_type* s2, size_t n);
  static char_type* copy(char_type* s1, const char_type* s2, size_t n);
  static char_type* assign(char_type* s, size_t n, char_type a);
  static constexpr int_type not_eof(int_type c) noexcept;
  static constexpr char_type to_char_type(int_type c) noexcept;
  static constexpr int_type to_int_type(char_type c) noexcept;
  static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
  static constexpr int_type eof() noexcept;
 };
}
```

Drafting note: The `char_traits<char8_t>` specification above was copied from the `char_traits<char16_t>` specification in [char.traits.specializations.char16_t] and then modified to update the targets of the type aliases.

Add paragraph 1:

> The two-argument members `assign`, `eq`, and `lt` are defined identically to the built-in operators `=`, `==`, and `<` respectively.

Add paragraph 2:

> The member `eof()` returns an implementation-defined constant that cannot appear as a valid UTF-8 code unit.

Drafting note: Paragraphs 1-2 above are lightly edited copies from the `char_traits<char16_t>` specification in [char.traits.specializations.char16_t] that were then modified to match wording changes in Tim Song's proposed cleanup of the <string> library.

Change in 20.3 [string.classes] paragraph 1:

> The header `<string>` defines the `basic_string` class template for manipulating varying-length sequences of char-like objects and ~~four~~**five** *typedef-name*s, `string`, **`u8string`,** `u16string`, `u32string`, and `wstring`, that name the specializations `basic_string<char>`, **`basic_string<char8_t>`,** `basic_string<char16_t>`, `basic_string<char32_t>`, and `basic_string<wchar_t>`, respectively.

Change in 20.3.1 [string.syn]:

> **Header `<string>` synopsis**
>
> ```
> #include <initializer_list>
>
> namespace std {
>   // [char.traits], character traits:
>   template<class charT> struct char_traits;
>   template<> struct char_traits<char>;
>   template<> struct char_traits<char8_t>;
>   template<> struct char_traits<char16_t>;
>   template<> struct char_traits<char32_t>;
>   template<> struct char_traits<wchar_t>;
> […]
>   // basic_string typedef names
>   using string = basic_string<char>;
>   using u8string = basic_string<char8_t>;
> ```

```
    using u16string = basic_string<char16_t>;
    using u32string = basic_string<char32_t>;
    using wstring = basic_string<wchar_t>;
  […]
    namespace pmr {
      template <class charT, class traits = char_traits<charT>>
        using basic_string = std::basic_string<charT, traits, polymorphic_allocator<charT>>;

      using string = basic_string<char>;
      using u8string = basic_string<char8_t>;
      using u16string = basic_string<char16_t>;
      using u32string = basic_string<char32_t>;
      using wstring = basic_string<wchar_t>;
    }
  […]
    // [basic.string.hash], hash support:
    template<class T> struct hash;
    template<> struct hash<string>;
    template<> struct hash<u8string>;
    template<> struct hash<u16string>;
    template<> struct hash<u32string>;
    template<> struct hash<wstring>;

    template<> struct hash<pmr::string>;
    template<> struct hash<pmr::u8string>;
    template<> struct hash<pmr::u16string>;
    template<> struct hash<pmr::u32string>;
    template<> struct hash<pmr::wstring>;

    inline namespace literals {
    inline namespace string_literals {
      // [basic.string.literals], suffix for basic_string literals:
      string operator "" s(const char* str, size_t len);
      u8string operator "" s(const char8_t* str, size_t len);
      u16string operator "" s(const char16_t* str, size_t len);
      u32string operator "" s(const char32_t* str, size_t len);
      wstring operator "" s(const wchar_t* str, size_t len);
    }
    }
  }
```

Change in 20.3.5 [basic.string.hash]:

```
template<> struct hash<string>;
template<> struct hash<u8string>;
template<> struct hash<u16string>;
template<> struct hash<u32string>;
template<> struct hash<wstring>;
template<> struct hash<pmr::string>;
template<> struct hash<pmr::u8string>;
template<> struct hash<pmr::u16string>;
template<> struct hash<pmr::u32string>;
template<> struct hash<pmr::wstring>;
```

Add a new paragraph after 20.3.6 [basic.string.literals] paragraph 1:

```
u8string operator""s(const char8_t* str, size_t len);
  Returns: u8string{str, len}.
```

Change in 20.4.1 [string.view.synop]:

```
  […]
```

```
    // basic_string_view typedef names
    using string_view = basic_string_view<char>;
    using u8string_view = basic_string_view<char8_t>;
    using u16string_view = basic_string_view<char16_t>;
    using u32string_view = basic_string_view<char32_t>;
    using wstring_view = basic_string_view<wchar_t>;

    // [string.view.hash], hash support
    template<class T> struct hash;
    template<> struct hash<string_view>;
    template<> struct hash<u8string_view>;
    template<> struct hash<u16string_view>;
    template<> struct hash<u32string_view>;
    template<> struct hash<wstring_view>;

    inline namespace literals {
    inline namespace string_view_literals {
      // [string.view.literals], suffix for basic_string_view literals
      constexpr string_view operator""sv(const char* str, size_t len) noexcept;
      constexpr u8string_view operator""sv(const char8_t* str, size_t len) noexcept;
      constexpr u16string_view operator""sv(const char16_t* str, size_t len) noexcept;
      constexpr u32string_view operator""sv(const char32_t* str, size_t len) noexcept;
      constexpr wstring_view operator""sv(const wchar_t* str, size_t len) noexcept;
    }
    }
  […]
```

Change in 20.4.5 [string.view.hash]:

```
template<> struct hash<string_view>;
template<> struct hash<u8string_view>;
template<> struct hash<u16string_view>;
template<> struct hash<u32string_view>;
template<> struct hash<wstring_view>;
```

Add a new paragraph after 20.4.6 [string.view.literals] paragraph 1:

```
constexpr u8string_view operator""sv(const char8_t* str, size_t len) noexcept;
  Returns: u8string_view{str, len}.
```

Change in 20.5.5 [cuchar.syn]:

```
namespace std {
  using mbstate_t = see below;
  using size_t = see 16.2.4;

  size_t mbrtoc8(char8_t* pc8, const char* s, size_t n, mbstate_t* ps);
  size_t c8rtomb(char* s, char8_t c8, mbstate_t* ps);
  size_t mbrtoc16(char16_t* pc16, const char* s, size_t n, mbstate_t* ps);
  size_t c16rtomb(char* s, char16_t c16, mbstate_t* ps);
  size_t mbrtoc32(char32_t* pc32, const char* s, size_t n, mbstate_t* ps);
  size_t c32rtomb(char* s, char32_t c32, mbstate_t* ps);
}
```

Change in 20.5.5 [cuchar.syn] paragraph 1:

> The contents and meaning of the header <cuchar> are the same as the C standard library header <uchar.h>, except that it **declares the additional mbrtoc8 and c8rtomb functions, and** does not declare types char16_t nor char32_t.
>
> *See also:* ISO C 7.28

Drafting note: If WG14 were to adopt N2231 [WG14 N2231] in a future revision of ISO C, and if WG21 were to update its normative reference to ISO C to a later revision containing those changes, then the updates to 20.5.5 paragraph 1 above will require

modification to exclude a declaration of the `char8_t` typedef and to remove mention of the additional `mbrtoc8` and `c8rtomb` functions.

Change in 20.5.6 [c.mb.wcs] paragraph 1:

> *[Note: The headers <cstdlib> (16.2.2), **<cuchar> (20.5.5),** and <cwchar> (20.5.4) declare the functions described in this subclause. — end note]*

Add the following paragraphs at the end of 20.5.6 [c.mb.wcs]:

```
size_t mbrtoc8(char8_t* pc8, const char* s, size_t n, mbstate_t* ps);
```

7 *Effects:* If `s` is a null pointer, equivalent to

```
mbrtoc8(nullptr, "", 1, ps)
```

Otherwise, the function inspects at most `n` bytes beginning with the byte pointed to by `s` to determine the number of bytes needed to complete the next multibyte character (including any shift sequences). If the function determines that the next multibyte character is complete and valid, it determines the values of the corresponding UTF-8 code units and then, if `pc8` is not a null pointer, stores the value of the first (or only) such code unit in the object pointed to by `pc8`. Subsequent calls will store successive UTF-8 code units without consuming any additional input until all the code units have been stored. If the corresponding Unicode character is `U+0000`, the resulting state described is the initial conversion state.

8 *Returns:* The first of the following that applies (given the current conversion state):

(8.1)	`0`	if the next `n` or fewer bytes complete the multibyte character that corresponds to the `U+0000` Unicode character (which is the value stored).
(8.2)	between 1 and `n` inclusive	if the next `n` or fewer bytes complete a valid multibyte character (which is the value stored); the value returned is the number of bytes that complete the multibyte character.
(8.3)	`(size_t) (-3)`	if the next character resulting from a previous call has been stored (no bytes from the input have been consumed by this call).
(8.4)	`(size_t) (-2)`	if the next `n` bytes contribute to an incomplete (but potentially valid) multibyte character, and all `n` bytes have been processed (no value is stored).
(8.5)	`(size_t) (-1)`	if an encoding error occurs, in which case the next `n` or fewer bytes do not contribute to a complete and valid multibyte character (no value is stored); the value of the macro `EILSEQ` is stored in `errno`, and the conversion state is unspecified.

```
size_t c8rtomb(char* s, char8_t c8, mbstate_t* ps);
```

9 *Effects:* If `s` is a null pointer, equivalent to

```
c8rtomb(buf, u8'\0', ps)
```

where `buf` is an internal buffer. Otherwise, if `c8` completes a sequence of valid UTF-8 code units, determines the number of bytes needed to represent the multibyte character (including any shift sequences), and stores the multibyte character representation in the array whose first element is pointed to by `s`. At most `MB_CUR_MAX` bytes are stored. If the multibyte character is a null character, a null byte is stored, preceded by any shift sequence needed to restore the initial shift state; the resulting state described is the initial conversion state.

11 *Returns:* The number of bytes stored in the array object (including any shift sequences). If `c8` does not contribute to a sequence of `char8_t` corresponding to a valid multibyte character, the value of the macro `EILSEQ` is stored in `errno`, `(size_t) (-1)` is returned, and the conversion state is unspecified.

10 *Remarks:* Calls to `c8rtomb` with a null pointer argument for `buf` may introduce a data race (15.5.5.9) with other calls to `c8rtomb` with a null pointer argument for `buf`.

Drafting note: The wording for `mbrtoc8` and `c8rtomb` is derived from wording for `mbrtoc16` and `c16rtomb` in C18 (WG14 N2176), augmented by changes suggested in WG14 N2040 for WG14 DR488 to properly account for UTF-8 being a variable length encoding, and lightly edited for formatting style. The author was reluctant to stray from the existing C wording for related functions despite a belief that considerable improvements to the wording would be possible.

Change in table 91 of 26.3.1.1.1 [locale.category] paragraph 2:

Table 91 — Locale category facets

Category	Includes facets
[...]	[...]
ctype	`ctype<char>, ctype<wchar_t>`

	codecvt<char,char,mbstate_t> ~~codecvt<char16_t,char,mbstate_t>~~ ~~codecvt<char32_t,char,mbstate_t>~~ **codecvt<char16_t,char8_t,mbstate_t>** **codecvt<char32_t,char8_t,mbstate_t>** codecvt<wchar_t,char,mbstate_t>
[...]	[...]

Drafting note: The deleted `char` based `codecvt` specializations have been deprecated and moved to annex D, [depr.locale.category].

Change in table 92 of 26.3.1.1.1 [locale.category] paragraph 4:

Table 92 — Required specializations

Category	Includes facets
[...]	[...]
ctype	ctype_byname<char>, ctype_byname<wchar_t> codecvt_byname<char,char,mbstate_t> ~~codecvt_byname<char16_t,char,mbstate_t>~~ ~~codecvt_byname<char32_t,char,mbstate_t>~~ **codecvt_byname<char16_t,char8_t,mbstate_t>** **codecvt_byname<char32_t,char8_t,mbstate_t>** codecvt_byname<wchar_t,char,mbstate_t>
[...]	[...]

Drafting note: The deleted `char` based `codecvt_byname` specializations have been deprecated and moved to annex D, [depr.locale.category].

Change in 26.4.1.4 [locale.codecvt] paragraph 3:

> The specializations required in Table 91 (26.3.1.1.1) convert the implementation-defined native character set. `codecvt<char, char, mbstate_t>` implements a degenerate conversion; it does not convert at all. The specialization `codecvt<char16_t, ~~char~~char8_t, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specialization `codecvt<char32_t, ~~char~~char8_t, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms. `codecvt<wchar_t,char,mbstate_t>` converts between the native character sets for ~~narrow~~**ordinary** and wide characters. Specializations on `mbstate_t` perform conversion between encodings known to the library implementer. Other encodings can be converted by specializing on a user-defined `stateT` type. Objects of type `stateT` can contain any state that is useful to communicate to or from the specialized `do_in` or `do_out` members.

Change in 27.3.1 [iosfwd.syn]:

```
[…]
  template<class charT> class char_traits;
  template<> class char_traits<char>;
  template<> class char_traits<char8_t>;
  template<> class char_traits<char16_t>;
  template<> class char_traits<char32_t>;
  template<> class char_traits<wchar_t>;
[…]
  template<class state> class fpos;
  using streampos = fpos<char_traits<char>::state_type>;
  using wstreampos = fpos<char_traits<wchar_t>::state_type>;
  using u8streampos = fpos<char_traits<char8_t>::state_type>;
[…]
```

Change in 27.11.4 [fs.req] paragraph 1:

> Throughout this subclause, `char`, `wchar_t`, **`char8_t`,** `char16_t`, and `char32_t` are collectively called *encoded character types*.

Change in 27.11.5 [fs.filesystem.syn]:

```
  ~~// 27.11.7.7.1, path factory functions~~
```

```
~~template <class Source>~~
~~  path u8path(const Source& source);~~
~~template <class InputIterator>~~
~~  path u8path(InputIterator first, InputIterator last);~~
```

Drafting note: The deleted `u8path` factory functions have been deprecated and moved to annex D, [depr.fs.path.factory].

Change in 27.11.7 [fs.class.path] paragraph 6:

```
[…]
  std::string string() const;
  std::wstring wstring() const;
  std::~~string~~u8string u8string() const;
  std::u16string u16string() const;
  std::u32string u32string() const;
[…]
  std::string generic_string() const;
  std::wstring generic_wstring() const;
  std::~~string~~u8string generic_u8string() const;
  std::u16string generic_u16string() const;
  std::u32string generic_u32string() const;
[…]
```

Change in 27.11.7.2.2 [fs.path.type.cvt] paragraph 1:

> The *native encoding* of ~~a narrow~~**an ordinary** character string is the operating system dependent current encoding for pathnames (27.11.7). The *native encoding* for wide character strings is the implementation-defined execution wide-character set encoding (5.3).

Change in 27.11.7.2.2 [fs.path.type.cvt] subparagraph (2.1):

> (2.1) — `char`: The encoding is the native ~~narrow~~**ordinary** encoding. The method of conversion, if any, is operating system dependent. [*Note*: For POSIX-based operating systems `path::value_type` is `char` so no conversion from `char` value type arguments or to `char` value type return values is performed. For Windows-based operating systems, the native ~~narrow~~**ordinary** encoding is determined by calling a Windows API function. — *end note*] [*Note*: This results in behavior identical to other C and C++ standard library functions that perform file operations using ~~narrow~~**ordinary** character strings to identify paths. Changing this behavior would be surprising and error prone. — *end note*]

Add a new subparagraph after 27.11.7.2.2 [fs.path.type.cvt] subparagraph (2.2):

> (2.?) — `char8_t`: The encoding is UTF-8. The method of conversion is unspecified.

Change in 27.11.7.4.1 [fs.path.construct] subparagraph (7.2):

> — Otherwise a conversion is performed using the `codecvt<wchar_t, char, mbstate_t>` facet of `loc`, and then a second conversion to the current ~~narrow~~**ordinary** encoding.

Drafting note: Is the requirement for a second conversion stated above correct? `codecvt<wchar_t, char, mbstate_t>` already converts to the ordinary character encoding.

Change in 27.11.7.4.1 [fs.path.construct] paragraph 8:

> […]
> For POSIX-based operating systems, the path is constructed by first using `latin1_facet` to convert ISO/IEC 8859-1 encoded `latin1_string` to a wide character string in the native wide encoding (27.11.7.2.2). The resulting wide string is then converted to ~~a narrow~~**an ordinary** character pathname string in the current native ~~narrow~~**ordinary** encoding. If the native wide encoding is UTF-16 or UTF-32, and the current native ~~narrow~~**ordinary** encoding is UTF-8, all of the characters in the ISO/IEC 8859-1 character set will be converted to their Unicode representation, but for other native ~~narrow~~**ordinary** encodings some characters may have no representation. […]

Change in 27.11.7.4.6 [fs.path.native.obs] paragraph 8:

```
  std::string string() const;
  std::wstring wstring() const;
```

```
  std::stringu8string u8string() const;
  std::u16string u16string() const;
  std::u32string u32string() const;

```

Returns: `native()`.

Change in 27.11.7.4.6 [fs.path.native.obs] paragraph 9:

Remarks: Conversion, if any, is performed as specified by 27.11.7.2. ~~The encoding of the string returned by `u8string()` is always UTF-8.~~

Change in 27.11.7.4.7 [fs.path.generic.obs] paragraph 5:

```
  std::string generic_string() const;
  std::wstring generic_wstring() const;
  std::stringu8string generic_u8string() const;
  std::u16string generic_u16string() const;
  std::u32string generic_u32string() const;

```

Returns: The pathname in the generic format.

Change in 27.11.7.4.7 [fs.path.generic.obs] paragraph 6:

Remarks: Conversion, if any, is specified by 27.11.7.2. ~~The encoding of the string returned by `generic_u8string()` is always UTF-8.~~

Remove subclause 27.11.7.7.1 [fs.path.factory].

~~`template<class Source>`~~
~~`  path u8path(const Source& source);`~~
~~`template<class InputIterator>`~~
~~`  path u8path(InputIterator first, InputIterator last);`~~

1 *Requires:* The source and `[first, last)` sequences are UTF-8 encoded. The value type of `Source` and `InputIterator` is `char`.

2 *Returns:*

(2.1) — If `value_type` is `char` and the current native narrow encoding (27.11.7.2.2) is UTF-8, return `path(source)` or `path(first, last)`; otherwise,

(2.2) — if `value_type` is `wchar_t` and the native wide encoding is UTF-16, or if `value_type` is `char16_t` or `char32_t`, convert `source` or `[first, last)` to a temporary, `tmp`, of type `string_type` and return `path(tmp)`; otherwise,

(2.3) — convert `source` or `[first, last)` to a temporary, `tmp`, of type `u32string` and return `path(tmp)`.

3 *Remarks:* Argument format conversion (27.11.7.2.1) applies to the arguments for these functions. How Unicode encoding conversions are performed is unspecified.

[*Example:* A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
  namespace fs = std::filesystem;
  std::string utf8_string = read_utf8_data();
  fs::create_directory(fs::u8path(utf8_string));
```

4 For POSIX-based operating systems with the native narrow encoding set to UTF-8, no encoding or type conversion occurs.
For POSIX-based operating systems with the native narrow encoding not set to UTF-8, a conversion to UTF-32 occurs, followed by a conversion to the current native narrow encoding. Some Unicode characters may have no native character set representation.
For Windows-based operating systems a conversion from UTF-8 to UTF-16 occurs. — *end example*]

Drafting note: The `u8path` factory function templates have been deprecated and moved to annex D, [depr.fs.path.factory].

Change in 29.2 [atomics.syn]:

```
[…]
  // [atomics.lockfree], lock-free property
  #define ATOMIC_BOOL_LOCK_FREE unspecified
  #define ATOMIC_CHAR_LOCK_FREE unspecified
  #define ATOMIC_CHAR8_T_LOCK_FREE unspecified
  #define ATOMIC_CHAR16_T_LOCK_FREE unspecified
  #define ATOMIC_CHAR32_T_LOCK_FREE unspecified
  #define ATOMIC_WCHAR_T_LOCK_FREE unspecified
[…]
  using atomic_ullong   = atomic<unsigned long long>;
  using atomic_char8_t  = atomic<char8_t>;
  using atomic_char16_t = atomic<char16_t>;
  using atomic_char32_t = atomic<char32_t>;
  using atomic_wchar_t  = atomic<wchar_t>;
```

Change in 29.5 [atomics.lockfree]:

```
  #define ATOMIC_BOOL_LOCK_FREE unspecified
  #define ATOMIC_CHAR_LOCK_FREE unspecified
  #define ATOMIC_CHAR8_T_LOCK_FREE unspecified
  #define ATOMIC_CHAR16_T_LOCK_FREE unspecified
  #define ATOMIC_CHAR32_T_LOCK_FREE unspecified
  #define ATOMIC_WCHAR_T_LOCK_FREE unspecified
  […]
```

Change in 29.6.2 [atomics.ref.int] paragraph 1:

> There are specializations of the `atomic_ref` class template for the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, **`char8_t,`** `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header <cstdint>. […]
> […]

Change in 29.7.2 [atomics.types.int] paragraph 1:

> There are specializations of the `atomic` class template for the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, **`char8_t,`** `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header <cstdint>. […]
> […]

Annex A Grammar summary wording

Change in A.6 [gram.dcl]:

```
[…]
simple-type-specifier:    […]
   char
   char8_t
   char16_t
   char32_t
   wchar_t
   […]
[…]
```

Annex C Compatibility wording

Change in C.1.1 [diff.lex] paragraph 3:

> […]
> **Affected subclause:** 5.13.5
> **Change**: String literals made const.
> The type of a string literal is changed from "array of `char`" to "array of `const char`". **The type of a UTF-8 string**

literal is changed from "array of `char`" to "array of `const char8_t`". The type of a `char16_t` string literal is changed from "array of *some-integer-type*" to "array of `const char16_t`". The type of a `char32_t` string literal is changed from "array of *some-integer-type*" to "array of `const char32_t`". The type of a wide string literal is changed from "array of `wchar_t`" to "array of `const wchar_t`".
[...]

Change in C.5.1 [diff.cpp17.lex] paragraph 1:

Affected subclause: 5.11
Change: New keywords
Rationale: Required for new features. The `requires` keyword is added to introduce constraints through a *requires-clause* or a *requires-expression*. The `concept` keyword is added to enable the definition of *concepts* (12.6.8). **The `char8_t` keyword is added to differentiate the types of ordinary and UTF-8 literals (5.13.5).**
Effect on original feature: Valid ISO C++ 2017 code using `concept`, ~~or~~ `requires`, **or `char8_t`** as an identifier is not valid in this International Standard.

Add a new paragraph to C.5.1 [diff.cpp17.lex]:

Affected subclause: 5.13
Change: Type of UTF-8 string and character literals.
Rationale: Required for new features. The changed types enable function overloading, template specialization, and type deduction to distinguish ordinary and UTF-8 string and character literals.
Effect on original feature: Valid ISO C++ 2017 code that depends on UTF-8 string literals having type "array of `const char`" and UTF-8 character literals having type "`char`" is not valid in this International Standard.

```
const auto *u8s = u8"text";    // u8s previously deduced as const char *; now deduced as const char8_t *.
const char *ps = u8s;          // ill-formed; previously well-formed.

auto u8c = u8'c';              // u8c previously deduced as char; now deduced as char8_t.
char *pc = &u8c;               // ill-formed; previously well-formed.

std::string s = u8"text";      // ill-formed; previously well-formed.

void f(const char *s);
f(u8"text");                   // ill-formed; previously well-formed.

template<typename> struct ct;
template<> struct ct<char> {
  using type = char;
};
ct<decltype(u8'c')>::type x;   // ill-formed; previously well-formed.
```

Add a new subclause after C.5.8 [diff.cpp17.containers]:

C.5.? [input.output]: Input/output library [diff.cpp17.input.output]

Affected subclause: 27.7.5.2.4
Change: Overload resolution for ostream inserters used with UTF-8 literals.
Rationale: Required for new features.
Effect on original feature: Valid ISO C++ 2017 code that passes UTF-8 literals to `basic_ostream::operator<<` no longer calls character related overloads.

```
std::cout << u8"text";        // Previously called operator<<(const char*) and printed a string.
                              // Now calls operator<<(const void*) and prints a pointer value.
std::cout << u8'X';           // Previously called operator<<(char) and printed a character.
                              // Now calls operator<<(int) and prints an integer value.
```

Affected subclause: 27.11.7
Change: Return type of filesystem path format observer member functions.
Rationale: Required for new features.
Effect on original feature: Valid ISO C++ 2017 code that depends on the `u8string()` and `generic_u8string()` member functions of `std::filesystem::path` returning `std::string` is not valid in this International Standard.

```
std::filesystem::path p;
std::string s1 = p.u8string();          // ill-formed; previously well-formed.
std::string s2 = p.generic_u8string();  // ill-formed; previously well-formed.
```

Annex D Compatibility features wording

Add a new subclause after D.14 [depr.conversions]:

D.?? Deprecated locale category facets [depr.locale.category]

The `ctype` locale category includes the following facets as if they were specified in table 91 of 26.3.1.1.1.

1
```
codecvt<char16_t, char, mbstate_t>
codecvt<char32_t, char, mbstate_t>
```

The `ctype` locale category includes the following facets as if they were specified in table 92 of 26.3.1.1.1.

2
```
codecvt_byname<char16_t, char, mbstate_t>
codecvt_byname<char32_t, char, mbstate_t>
```

3 The following class template specializations are required in addition to those specified in [locale.codecvt]. The specialization `codecvt<char16_t, char, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specialization `codecvt<char32_t, char, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms.

Add another new subclause after D.14 [depr.conversions]:

D.?? Deprecated filesystem path factory functions [depr.fs.path.factory]

The header `<filesystem>` has the following additions:

1
```
namespace std::filesystem {
  template <class Source>
    path u8path(const Source& source);
  template <class InputIterator>
    path u8path(InputIterator first, InputIterator last);
}
```

2 *Requires:* The `source` and `[first, last)` sequences are UTF-8 encoded. The value type of `Source` and `InputIterator` is `char`. `Source` meets the requirements specified in 27.11.7.3.

3 *Returns:*

(3.1) — If `path::value_type` is `char` and the current native narrow encoding (27.11.7.2.2) is UTF-8, return `path(source)` or `path(first, last)`; otherwise,

(3.2) — if `path::value_type` is `wchar_t` and the native wide encoding is UTF-16, or if `path::value_type` is `char16_t` or `char32_t`, convert `source` or `[first, last)` to a temporary, `tmp`, of type `path::string_type` and return `path(tmp)`; otherwise,

(3.3) — convert `source` or `[first, last)` to a temporary, `tmp`, of type `u32string` and return `path(tmp)`.

4 *Remarks:* Argument format conversion applies to the arguments for these functions. How Unicode encoding conversions are performed is unspecified.

[*Example:* A string is to be read from a database that is encoded in UTF-8, and used to create a directory using the native encoding for filenames:

```
namespace fs = std::filesystem;
std::string utf8_string = read_utf8_data();
fs::create_directory(fs::u8path(utf8_string));
```

5 For POSIX-based operating systems with the native narrow encoding set to UTF-8, no encoding or type conversion occurs.
For POSIX-based operating systems with the native narrow encoding not set to UTF-8, a conversion to UTF-32

occurs, followed by a conversion to the current native narrow encoding. Some Unicode characters may have no native character set representation.
For Windows-based operating systems a conversion from UTF-8 to UTF-16 occurs. — *end example*]

Drafting note: The contents of paragraph 1 correspond to the text removed from [fs.filesystem.syn]. The contents of paragraphs 2-5 correspond to the text removed from [fs.path.factory]

Acknowledgements

Michael Spencer and Davide C. C. Italiano first proposed adding a new `char8_t` fundamental type in P0372R0 [P0372R0].

Thanks to Alisdair Meredith for reviewing wording and providing feedback in advance of the Rapperswil meeting. Thanks to Tim Song and Casey Carter for further "paper of the week" wording review prior to San Diego.

References

[W3Techs] "Usage of UTF-8 for websites", W3Techs, 2017.
https://w3techs.com/technologies/details/en-utf8/all/all

[N2249] Lawrence Crowl, "New Character Types in C++", N2249, 2007.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2249.html

[N4197] Richard Smith, "Adding u8 character literals", N4197, 2014.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4197.html

[N4762] "Working Draft, Standard for Programming Language C++", N4762, 2018.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf

[P0372R0] Michael Spencer and Davide C. C. Italiano, "A type for utf-8 data", P0372R0, 2016.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0372r0.html

[P0244R2] Tom Honermann, "Text_view: A C++ concepts and range based character encoding and code point enumeration library", P0244R2, 2017.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0244r2.html

[P0218R1] Beman Dawes, "Adopt the File System TS for C++17", P0218R1, 2016.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0372r0.html

[WG14 N2231] Tom Honermann, "char8_t: A type for UTF-8 characters and strings", WG14 N2231, 2018.
http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2231.htm